

Explanation as Architecture: Recorded Traces, Provenance, and Testable Diagnostics in Compiler Construction

Lojan Essam ElDin Farouk

lojanessamfarouk@gmail.com

ABSTRACT. Compilers routinely discard the evidence behind their own diagnostics: the analysis knew which function it was checking, which grammar rule it was inside, and which check failed, but what survives to the user is a rendered string and a location. We present an architecture that treats explanation as a first-class obligation of every pipeline stage rather than a presentation-layer afterthought. Every diagnostic is a structured artifact carrying a trace *recorded from the compiler's actual execution*; all explanatory prose lives in a root-cause-indexed knowledge base isolated from the stages; source provenance survives lowering and optimization into the emitted assembly; and diagnostic-quality properties — cascade-freedom, trace accuracy — are enforced as executable tests. We validate the architecture in a complete, zero-dependency compiler for a C-like kernel language (~5.9K LOC, 205 tests) and measure its costs factorially: naive always-on trace recording cost $2.4\times$ compile time and was repaired to statistical insignificance by lazy detail formatting; provenance costs a steady 12–17%, attributable to per-instruction metadata growth (116→160 bytes); peak memory is unaffected. We report negative results as findings, including the non-transitivity of per-instruction provenance under dead-code elimination. The term "explanation" here denotes preserved compiler evidence, not machine-learning explainability, and no claims about human benefit are made without the user study we design but do not run.

§3 Introduction

During this compiler's development, its semantic analyzer detected a program defect and told no one. The program was `int main() { int x = 5; }` — a non-void function that never returns. The analyzer's internal log recorded the failure precisely ("bare return in non-void function"); no diagnostic was ever constructed from that knowledge; compilation "succeeded"; and the produced binary returned whatever value happened to occupy the return register. The compiler knew. The user didn't. We found the defect not by reading the code but by running twenty adversarial programs against the finished compiler — and we report it as the paper's motivating observation because it is the general phenomenon in miniature: **the evidence behind compiler diagnostics is routinely discarded before, or instead of, reaching the user.** What a conventional pipeline retains at the point of failure — a message string and a source location — is a small projection of what it knew moments earlier: the construct being analyzed, the rule being applied, the path taken to get there, and what became of the user's code in every stage downstream.

This is not the familiar complaint that error messages are unhelpful. Message *quality* has received sustained attention in production compilers — Rust's structured suggestions [3], GHC's migration to errors-as-values [4] [→§5] — and a substantial research literature examines message effectiveness, with notably contested results: controlled studies of *enhanced* messages report effects ranging from

ineffectual to inconclusive [1, 2]. We take that contestation seriously: this paper makes no claim that its diagnostics help users more, and the user study such a claim requires is designed but not run (§13). Our question is upstream of wording, and architectural: **what must a compiler retain, and where must it retain it, for explanation to be possible at all — and what does retaining it cost?**

Two scope commitments frame everything that follows. First, the vehicle is a deliberately small compiler: a C-like kernel language (integers, arithmetic, functions without calls) compiled through a complete pipeline — lexing, recursive-descent parsing, semantic analysis, three-address IR, three optimization passes run to a fixed point, and 32-bit x86 code generation to a linked executable. The contribution is the architecture and its measured costs, not a production technique; scaling questions are discussed, not claimed away (§10). Second, "explanation" here means *preserved compiler evidence* — recorded execution paths, provenance, root causes — not post-hoc rationalization, and not machine-learning explainability.

The architecture rests on one obligation: no stage may discard the evidence a future explanation would need. Concretely (Fig. F1 shows one diagnostic carrying all of it): every stage returns its product *and* its diagnostics, so partial results survive errors; a diagnostic is a structured value — kind, severity, span, root cause, explanation, fixes, examples, trace — never a string; all prose lives in one knowledge base indexed by root cause, so the taxonomy is

the explanation index; each stage owns a trace recorder whose RAII scopes capture the *actual* execution path, snapshotted at the moment a diagnostic is created; every IR instruction carries its source span and accumulates notes from the optimizer passes that transform it, surfacing as source-line comments in the emitted assembly; and the properties that make diagnostics trustworthy — one mistake yields one diagnostic; traces contain runtime facts — are regression tests, not aspirations.

Making that architecture honest required measuring it, and the measurements pushed back. Our first overhead campaign measured the harness, not the compiler, and was discarded (§9.2 explains how, so readers do not repeat it). The second refuted our own hypothesis: naive trace recording — eagerly building detail strings at every scope entry — cost 2.4× compile time at 20k lines, dominated by two scopes per token in the lexer. The repair (lazy detail formatting over caller-owned data, exploiting the fact that snapshots occur only while frames are open) brought recording to 1–2% with confidence intervals spanning zero, with byte-identical output. Provenance, by contrast, is *not* free: a steady 12–17%, mechanical in origin — the span and note fields grow every IR instruction from 116 to 160 bytes, and every copy and traversal pays. We also report what provenance cannot promise: per-instruction history is not transitive under dead-code elimination (a folded instruction's history dies with it once propagation makes it dead), and our cascade-freedom invariants, while holding on every tested shape, leave a measured residual — 3% of single-fault mutants still produce three or more diagnostics.

This paper contributes:

- **C1 — an architectural pattern for explanation-oriented compilation:** the five-role diagnostic core (root-cause taxonomy, factory, isolated knowledge base, multi-target rendering) over a pipeline-wide output-plus-diagnostics contract — presented as the distilled, designed-in form of a structure production compilers (GHC, Rust) have been converging toward by retrofit [3, 4] [→§5].
- **C2 — execution-recorded diagnostic traces:** RAII scope recording with snapshot-at-creation and a curated fallback that is provably unreachable in the pipeline; with the lazy-formatting design that makes always-on recording statistically free, and the measurements for both designs.
- **C3 — end-to-end provenance under the same contract:** source spans and transformation notes surviving to emitted assembly — conceding LLVM's optimization remarks [5] as production-scale prior art for the mechanism, and contributing its unification with

the diagnostic architecture, its measured price, and the DCE non-transitivity finding.

- **C4 — diagnostic quality as executable invariants:** property- level tests (cascade-freedom, trace accuracy) distinct from golden-output snapshots, with mutation testing quantifying the frontier they do not yet cover.

§4 fixes vocabulary and the language; §5 positions against five literature clusters; §6 presents the architecture as fourteen design decisions, each argued against a rejected alternative; §7 describes the visualization consumer; §8 the implementation; §9 the three-iteration measurement campaign and quality studies; §§10–13 discuss scaling, validity, limitations, and future work, including the designed user study; §14 concludes.

ERROR TYPE:	LOCATION:	TRACE:
Malformed Expression	line 3, column 14	Parser
	3 return x 2;	-> Parser::p
	^	-> Parser::i
ROOT CAUSE:		[start:
Two expressions appeared	INVALID: x 2	-> Parser::i
consecutively with no	VALID: x + 2	-> Parser::i
operator between them.	x * 2	[at 're
	x = 2	-> Parser::i
WHAT HAPPENED:		[at 're
The parser successfully		-> Diagnost
read: return x	HOW TO FIX:	
After completing that	Insert an operator	malform
expression, it encountered	between x and 2	[diagno
another value: 2	or remove the extra value	
C++ requires an operator		
between values.		

Figure 1. One diagnostic, verbatim (condensed two-column arrangement of the CLI's sequential output). Every field is data on one struct (§6); the trace is recorded from the parser's actual descent with runtime details in brackets.

§4 Background

4.1 Vocabulary

Four terms carry precise meanings in this paper. A **diagnostic** is a structured value describing one user-source mistake: kind, severity, source span, message, root cause, explanation, ordered fixes, optional invalid/valid examples, and a trace. A **trace** is the sequence of compiler activities that led to a diagnostic's creation; this paper distinguishes *recorded* traces (snapshots of the frames actually open at creation time) from *curated* ones (hand-written reference chains — the fallback, §6.3). **Provenance** is the backward link from a compiled artifact to its origin: which source line an IR instruction or assembly block derives from, and which optimization decisions transformed it. **Explanation** is the umbrella for all of the above — preserved compiler evidence, not machine-learning explainability and not post-hoc rationalization.

4.2 The source language

The vehicle is a C-like kernel language — deliberately not "a C++ subset" in any meaningful sense, though its programs are valid C. The complete grammar (verbatim from the parser's specification comment):

```
program      → function_decl* EOF
function_decl → type IDENT '(' param_list ')' block
param_list   → (type IDENT (',' type IDENT)*) | ε
block        → '{' statement* '}'
statement    → var_decl | return_stmt
var_decl     → type IDENT ('=' expression)? ';'
return_stmt  → 'return' expression? ';'
expression   → assignment
assignment   → IDENTIFIER '=' assignment | equality
equality     → comparison (('=='|'!=') comparison)*
comparison   → addition (('<'|'>') addition)*
addition     → multiply (('+'|'-') multiply)*
multiply     → unary (('*'|'/') unary)*
unary        → primary
primary      → INTEGER_LITERAL | IDENTIFIER | '(' expression ')'
type         → 'int'
```

Table T1 (feature inventory): **supported** — int variables with optional initializers; arithmetic and comparison operators with C precedence; assignment as a right-associative expression (`a = b = c`); multi-parameter functions; multi-error reporting with panic-mode recovery. **Absent by design** — control flow, function calls, strings, arrays, pointers, unary operators, additional types. Internally the type lattice contains `int/bool/unknown` (`bool` arises only as a comparison result; `unknown` is the error-recovery type that suppresses cascading type errors) and `void` (reserved).

The smallness is the method, not a concession: every architectural mechanism must be visible whole, every diagnostic kind enumerable (seventeen, of which five are explicitly reserved-unreachable), and every claim checkable against a codebase a reviewer can actually read (~5.9K lines, §8).

4.3 The pipeline

Compilation proceeds through eight stages: lexing (single-pass, maximal-munch), recursive-descent parsing to an AST whose nodes are pure data traversed by four independent visitors, semantic analysis (scope-stack symbol table; type resolution annotating the AST), lowering to a flat three-address IR, three optimization passes (constant folding, copy propagation, dead-code elimination) run together to a fixed point, code generation to 32-bit x86 assembly text, and an OS-facing driver that assembles and links a real executable via the system toolchain. The construction follows standard practice

throughout [29]; nothing in the pipeline itself is claimed as a contribution. What §6 adds is the explanation obligation threaded through every one of these stages — and §9 the bill for it.

§5 Related Work

Five clusters position this work; the two concessions come first, because a reviewer should meet them in our words before theirs.

5.1 Structured diagnostics in production compilers

Production compilers have been converging on diagnostics-as-data by retrofit. Rust formalized its error format and structured suggestions in RFC 1644 [3], emits machine-consumable JSON diagnostics [12], and maintains Fluent-based message infrastructure with a public error index [10]. GHC's migration from `SDoc` strings to "errors as structured values" — with `diagnosticReason` and `diagnosticHints` fields and public error codes since 9.6.1 — is documented across its development tracker and implementers' reports [4, 9], and the community maintains an error index [8]. **Concession one:** the five-role core of §6.2 is therefore not a new invention but a distillation; our claims are that designing it in from the first commit is cheap (§8), that the isolated knowledge base and root-cause-granularity taxonomy are the load-bearing choices (§6, D3–D4), and that the retrofit pain documented by these projects [4, 9] is evidence the pattern matters. To our knowledge, none of these systems provides the recorded per-diagnostic execution trace of §6.3. That knowledge is not passive: a targeted counterexample sweep (recorded in the reference registry) found only adjacent work — trace tooling for *student programs* [30], and compiler-internal debug dumps aimed at the compiler's own developers rather than its users — and if later work surfaces, this sentence and C2 narrow accordingly.

5.2 Optimization transparency and provenance

LLVM's remarks infrastructure [5] is production-scale prior art for per-transformation records: passes emit typed remark objects with source locations, serialized to YAML and visualized with dedicated tools. **Concession two:** our per-instruction notes are the same mechanism at teaching scale. The residue we claim: unification of provenance with the *diagnostic* architecture under one contract (one struct family, one renderer discipline, one JSON export), its measured always-on price (12–17%, §9.2 — LLVM's remarks are opt-in per build), the human-readable in-text form (# line N: comments in the emitted assembly, by analogy to `.loc` directives but for readers), and the negative

result that per-instruction history is not transitive under dead-code elimination (§6.4, §9).

5.3 Error-message quality and its contested effectiveness

A half-century of work examines programming error messages; Becker et al.'s working-group report maps the landscape [14]. Eye-tracking evidence shows developers genuinely read error messages and that reading difficulty predicts task performance [15]; design guidance for novice-facing messages continues to develop [16]. Yet controlled studies of *enhanced* messages report effects from ineffectual [1] to inconclusive [2] — a contested record extending into the LLM era, where generated explanations show promise in some deployments [25] and ineffectiveness in others [26]. This contestation shapes our claim discipline (§9.3): we measure component *presence* with a mechanical instrument descended from Marceau et al.'s evaluation rubric [13], and defer all helpfulness claims to a designed-but-unexecuted study (§13). Our contribution is orthogonal to wording quality: an architecture that *retains the evidence* any wording — human-authored or LLM-generated [25] — would need.

5.4 Error localization and interrogative debugging

Constraint-based localization computes *where* the likely mistake is: general static-error diagnosis via Bayesian-ranked unsatisfiable constraints [17, 18], type-error diagnosis [19], and SMT-based localization [20]. These are complementary: they improve one field of our diagnostic (the span and root-cause selection), while our architecture governs how the whole explanation is assembled, preserved, and delivered. The Whyline [11, 21] pioneered why/why-not questions over *program* executions via tracing and slicing, a family that extends into education through trace-divergence debugging of student code [30]; we ask the interrogative question of the *compiler's own run* — a far shallower trace, recorded live at negligible cost (§9.2) rather than reconstructed by post-hoc analysis.

5.5 Compiler visualization in education

The lineage is old — the UW Illustrated Compiler graphically displayed its own control and data structures to students in 1988 [28] — and continues through AST construction animators [22], interactive stage simulations [23], and, recently, JIT IR-evolution explorers [24]. Our visualizer (§7) sits in this lineage as a *consumer* of the architecture rather than a contribution: its distinguishing property is that every panel renders the same structured values the CLI renders, because the architecture guarantees a single source of truth (§6.2, D6).

Positioning summary (Table T2)

Capability	GHC [4]	rustc [3, 10, 12]	LLVM [5]	this work
Structured diagnostics as values	retrofit	retrofit	—	designed-in
Root-cause-indexed public error index	✓ [8]	✓	—	✓ (taxonomy = index, D3)
Machine-readable diagnostic export	✓	✓ [12]	✓ (remarks)	✓
Per-diagnostic recorded execution trace	—	—	—	✓ (§6.3)
Optimization provenance records	—	—	✓ opt-in [5]	✓ always-on, unified, priced
Diagnostic-quality property tests	snapshot-style	golden .stderr [10]	—	property-level + mutation-measured (§9)

§6 The Explanation Architecture

The architecture rests on a single obligation: *no stage may discard the evidence a future explanation would need*. This section presents the four mechanism groups that discharge that obligation — the stage contract (§6.1), the diagnostic core (§6.2), recorded traces (§6.3), and cross-stage provenance (§6.4) — each as an engineering decision weighed against at least one rejected alternative.

6.1 Stage contract and the taxonomy of failure

Every pipeline stage returns `StageOutput<T>`: its product *and* a list of structured diagnostics, always both. A stage that finds errors still returns its partial product; the driver, not the stage, decides whether to continue.

D1 — Partial-success contract vs. fail-fast propagation. The conventional alternatives are exceptions or an `Either`-style short-circuit (`Result<T,E>` end to end). We rejected both for the *pipeline* path because explanation quality depends on surviving the first error: a fail-fast parser cannot report the second, independent mistake, and — more damaging for this architecture — it discards the partially built AST that later explanation surfaces (the visualizer deliberately renders the recovered-but-erroneous tree, because *where the parser got confused*

is itself explanatory). The cost of the choice is real: every stage must define what "partial product" means (the parser synthesizes placeholder nodes; the analyzer types what it can and poisons the rest with `unknown`), and downstream code must tolerate it. Production compilers pay the same cost for the same reason — multi-error reporting has been standard since panic-mode recovery; see [6] for the modern treatment and its quantified failure mode, cascading. Evidence: `StageOutput<T>` in `src/diagnostics/Diagnostic.hpp`; the parser's recovery paths and the cascade-freedom tests they carry.

D2 — Three failure currencies vs. one. A uniform "everything is a `Diagnostic`" design was rejected because it conflates three audiences. User-source mistakes become `Diagnostic` (they have a `SourceSpan`; there is something to explain). Environment failures — assembler missing, disk full — become `Result<T,E>` (`src/driver/Toolchain.cpp`): they have no span, and routing them through the explanation machinery would manufacture explanations for facts about the machine, not the program. Violated *compiler invariants* are asserts (`src/ir/IRGenerator.cpp`): a bug in us, not in the user, and dressing it as a user-facing diagnostic would be a category error with a friendly font. The cost: three idioms to learn, and the boundary must be policed in review. The split mirrors LLVM's separation of recoverable errors ("an error in the program's environment... a missing file", handled via `Expected<T>/Error`) from its diagnostic subsystem [7]. Our own history enforces the boundary's value: the one failure that *was* misfiled (missing-return, logged internally as neither diagnostic nor error) produced the silent-garbage defect that motivates §3.

6.2 The five-role diagnostic core

A diagnostic here is one struct carrying kind, severity, span, message, root cause, explanation, ordered fixes, trace, and optional invalid/valid example pairs. Five roles cooperate to build and deliver it: a **kind taxonomy**, a **factory** (`DiagnosticEngine`), a **knowledge base** (`ExplanationBuilder`), a **collector** with three renderers, and the stage contract of §6.1 as transport.

D3 — One kind per root cause vs. one code per message. `DiagnosticKind` allocates an enumerator per *distinct underlying cause*, not per output string — `PARSE_MalformedExpression` covers both `return x 2`; and `int x = 5 5`; because the mistake (two expressions, no operator) is one cause with two surface forms. The granularity rule is operational, not aesthetic: two failures share a kind **iff** they share a root-cause sentence and a fix set — the knowledge base's own index structure decides. By contrast, a bare `return`; in a non-void function and a

`missing return` entirely get distinct kinds (`SEM_ReturnTypeMismatch`, `SEM_MissingReturn`) despite superficial kinship, because their causes, severities, and fixes all differ — one is an ill-formed statement (error), the other a missing obligation (warning, GCC's `-Wreturn-type-choice`). The rejected alternative, message-granularity codes, is the shape string-based systems drift into; GHC's and Rust's retrofits both converged on cause-indexed identifiers with public indexes [8, 4, 3]. Benefit: the taxonomy *is* the explanation index. Cost, honestly carried in the header: taxonomy discipline requires admitting unreachable entries; five kinds are marked `[reserved]` in `src/diagnostics/DiagnosticKind.hpp` because the language cannot yet produce them, and tests are forbidden from expecting them.

D4 — Centralized prose knowledge base vs. prose at the emit site vs. external message DSL. All user-facing explanation text lives in one component, `ExplanationBuilder`, switch-indexed by kind and parameterized by runtime detail. Emit-site prose (each stage writes its own sentences) was rejected because it makes consistency unenforceable and translation impossible — the exact pathology GHC documents in its `SDoc`-era retrofit discussion [9, 4]. The heavier alternative — an external message DSL (Clang's `TableGen .td` files [27]; rustc's `Fluent` [10]) — was rejected *at this scale*: it buys tooling and `i18n` at the price of a build step and an indirection layer, for seventeen kinds. The cost of our middle position surfaced measurably: the builder's `detail/detail2` interface hit its capacity limit the first time a kind needed three context values, and the factory works around it by re-deriving one field (`src/diagnostics/DiagnosticEngine.cpp`, documented at the `malformedExpression` site). We report that as evidence about where this design stops scaling; the industrial DSLs sit exactly where our workaround points (see Discussion, §10).

D5 — Factory/knowledge-base split vs. a merged emitter. `DiagnosticEngine` (runtime context in, complete struct out) is separate from `ExplanationBuilder` (static text per kind). A merged component would be smaller; the split was chosen because the two change for different reasons — one when *detection* context changes, one when *wording* changes — and because the knowledge base must stay stateless and stage-independent for the fallback semantics of §6.3 to be sound. Cost: one more hop to read, and a factory method per kind (mechanical boilerplate; sixteen methods at the time of writing).

D6 — Late rendering, three renderers vs. render-at-creation. Diagnostics are stored as data and rendered on demand — full explanatory text, compact one-liners, or JSON — from the same struct

(src/diagnostics/DiagnosticCollector.cpp). Rendering at creation (the string-concatenation norm this paper argues against) was rejected as the root discard-point: once prose, always prose. The measured consequence of late rendering is that CLI and visualizer provably tell one story (they consume the same struct; Fig. F6), and machine consumers get fields, not regexes — the property GHC's and Rust's JSON emitters retrofitted at cost [12, 4]. Cost: the struct is the compatibility surface; adding a field touches all three renderers (the trace detail field's addition did exactly that, in one commit).

6.3 Recorded traces

Every diagnostic carries the call path that produced it. The claim requiring defense is the word *recorded*.

D7 — RAII scope-stack snapshot vs. curated per-kind chains vs. full event log. The predecessor design — and this is committed history, not a rhetorical foil — was a *curated* chain: a hand-written, plausible call path per kind, stored in the knowledge base. It rendered convincingly and was architecturally cheap, but it could not name the function being analyzed or the token under the cursor, because those exist only at runtime; the paper's thesis calls this fabricated provenance. The replacement: each stage owns a `TraceRecorder`; methods at decision points open RAII scopes carrying runtime detail ("function 'main'", "at 'return' (line 3)"); when the factory builds a diagnostic it snapshots the frames *currently open* (src/diagnostics/TraceRecorder.hpp; Fig. F4 traces the sequence for the Fig. F1 diagnostic). The third option — logging every event, not just the live stack — was rejected on cost and noise grounds: an event log grows with program size whether or not an error ever occurs, while the active-stack snapshot is $O(\text{depth})$ and paid only at diagnostic creation; the Whyline demonstrates the power of full traces for *program* debugging and also their price, a dedicated post-hoc analysis apparatus [11]. Disclosed cost of our choice: sibling context is invisible (completed frames are gone from the stack). The entry cost of open frames is the subject of D14 and §9.2 — the naive version of this design failed its own overhead experiment. Evidence that recording is real rather than curated-with-extra-steps: a pre-existing regression test *failed* at the transition — it had encoded the curated chain's shape (`trace.front() = detecting method`), and recorded traces begin at the true outermost frame instead. The failure was the proof; the test now asserts containment.

D8 — Retaining the curated chains as fallback vs. deleting them. When an engine is used with no recorder attached — unit tests construct diagnostics directly — the knowledge base's static chain is substituted, and its final

step (the factory method) is appended to *recorded* chains too, so every trace terminates identically. Deleting the curated chains would have simplified the truth story at the cost of either dragging recorder setup into every unit test or letting traces go empty off-pipeline. We kept the fallback and disclose it. In the compiled pipeline the fallback is unreachable: exactly three `DiagnosticEngine` instances exist (one per diagnostic-emitting stage), and each attaches its recorder before parsing/analysis begins; IR generation, optimization, and code generation emit no diagnostics at all. Every diagnostic a user can ever see therefore carries a recorded trace (verified by exhaustive search for engine instantiations). The residual risk we accept is dual-source drift, mitigated by the shared final step and by tests that assert *recorded* traces contain runtime facts no static chain could hold.

D9 — Selective scope placement vs. instrument-everything. Placement follows a stated rule, not taste: a scope is opened (i) in every method that can itself emit a diagnostic, and (ii) at structural landmarks that name the enclosing construct (function, block, statement dispatch). Expression-precedence internals satisfy neither — their failures surface in `parsePrimary/expect`, which are covered by (i). A skeptic may object that selective scoping makes the recorded trace "curated by another name" — the author still chooses what is recordable. The objection conflates two different failure modes. A curated chain can be *wrong*: it asserts a path that did not execute (and before this transition, ours did — no curated chain could have named function 'main'). A selectively scoped recording can only be *coarse*: every frame it reports was genuinely open, verified by tests asserting runtime facts in the output. Omission bounds resolution; it cannot manufacture provenance. The placement policy is itself inspectable data (`grep TRACE_SCOPE`), and refining it is a local edit, not an architectural change.

D13 — Ablation by macro vs. no-op class (measurement honesty). The overhead experiments compare builds with recording compiled out. A no-op `TraceScope` class would still evaluate its argument expressions — the detail context built at every call site — and so would silently understate the mechanism's true cost. Call sites therefore use a `TRACE_SCOPE(...)` macro that erases its arguments entirely under `-DCPPC_NO_TRACE`. Cost: a macro where RAII would be idiomatic; accepted because a measurement that flatters the design is worse than an ugly line. (The same flag design gives the factorial attribution used in §9.2.)

D14 — Lazy detail formatting vs. eager strings (added after E1 refuted H3 for the eager design). The first recorder built each frame's detail *string* at scope

entry. E1 measured that at $\sim 2.4\times$ total compile time on 20k-line inputs — the lexer opens two scopes per token, so a clean compile paid for hundreds of thousands of string constructions whose text nobody would ever read. The repaired design stores lazily-formattable POD in each frame — `const char*` literals, a pointer to a caller-owned string, ints — and materializes text only in `snapshot()`, i.e. only when a diagnostic fires. Two correctness arguments carry the design: (1) *pointer stability* — snapshots only ever occur while frames are open (the engine runs inside the stage method that owns the scope), so a frame may safely point at a token's lexeme or an AST node's name, which provably outlive it; rvalue overloads are deleted so a temporary cannot be captured. (2) *Entry-time semantics* — ints (line/column) are copied by value at scope entry because the lexer's counters advance during the scope, and the frame must describe where the scope was *opened*, exactly as the eager strings did. One rare path (the parser's empty-lexeme case) needs text whose construction would couple the diagnostics layer to the lexer; it pays for an eager string via an explicit escape hatch rather than couple the layers. Alternatives rejected: deferred closures (a `std::function` per frame allocates — the disease as the cure) and fixed-size `snprintf` buffers (kills the allocation but keeps per-scope formatting work and truncates unpredictably). The materialized strings are byte-identical to the eager versions — the trace-content tests pin them, and passed unchanged across the transition. Measured effect (§9.2, interleaved protocol): recording cost fell from +143% [130, 168] to 1–2% with confidence intervals spanning zero at every corpus size — a $\sim 60\times$ reduction, leaving recording statistically indistinguishable from free while producing identical output.

6.4 Provenance to emitted assembly

D10 — Inline provenance fields vs. side tables vs. AST back-pointers. Every `IRInstruction` carries a `SourceSpan` and a transformation note as ordinary value fields, stamped at lowering (`src/ir/IRGenerator.cpp`). Back-pointers into the AST were rejected on lifetime grounds — they would couple IR validity to AST retention, which no later consumer otherwise needs. A side table (instruction $\rightarrow$ span) was rejected on *identity* grounds: optimizer passes rewrite instructions in place and rebuild instruction vectors (DCE), so any table keyed by identity or index desynchronizes precisely when provenance matters most; inline fields travel with the value through every rewrite for free. Constant folding's rewrite demonstrates the discipline the choice demands (quoted from `src/optimizer/ConstantFoldingPass.cpp`):

```
// Provenance: the replacement must keep the
// original's source span
// (it still comes from the same source expression)
// and record what
// it used to be — rebuilding via makeCopy() alone
// would silently
// wipe both.
IRValue      dest  = instr.dest;
SourceSpan   span  = instr.span;
std::string  before = instr.toString();
std::string  prior  = instr.note;
instr        =      IRInstruction::makeCopy(dest,
IRValue::makeConst(result));
instr.span = span; instr.note = prior;
instr.appendNote("folded from '" + before + "'
(ConstantFolding)");
```

The four saved locals are the cost of inline provenance made visible: every rewriting pass owes this discipline. A side table would trade those four lines for an identity-tracking problem across in-place rewrites and DCE's vector rebuild — a worse debt, but a real alternative for pass-heavy compilers. Costs of inline fields, both measured (§9.2): the enlarged instruction makes every copy, move, and pass traversal pay — a steady 12–17% wall time — while peak memory stays $\leq 1.3\%$; provenance's price is time, not space. And `toString()` had to stay byte-stable so thirty golden IR tests survived; provenance display therefore lives in separate channels (structured JSON, assembly comments) rather than in the canonical text.

D11 — Accumulated string notes vs. structured remark records. Passes append human-readable notes ("`t1 -> 20 (CopyPropagation)`"). The structured alternative is exactly what LLVM ships — typed remark objects serialized to YAML with dedicated viewers [5] — and we concede it is the scalable form. Strings were chosen because at this scale the consumer is a human reading a panel or a `.s` file, and a remark schema would be structure nothing downstream parses. The choice exposed a genuine finding the structured design would *also* exhibit: per-instruction history is **not transitive**. When DCE removes a folded instruction whose constant was already propagated onward, its `ConstantFolding` note dies with it; the surviving instruction reports only the propagation step (Fig. F5 walks the fixed-point iterations for $(2+3)*4$). Both the optimizer and visualizer suites lock this behavior in as documented semantics rather than accident — and it delimits what per-instruction provenance can ever promise without a dataflow-chained design (Future Work, §13).

D12 — Comment-based assembly provenance vs. debug-info directives. Codegen emits `# line N: <ir> ; <notes>` above each instruction block. The rejected alternative — `.loc/DWARF` — is the production answer and

is *machine*-consumable by debuggers; it was rejected here because our consumer is a reader, our 2006-era target toolchain makes DWARF plumbing a project of its own, and the paper's claim is about preserving explanation, not shipping debuggers. Cost: tools cannot consume our provenance from the binary; it lives in the text artifacts (assembly comments, JSON) only.

source (line 2): return (2 + 3) * 4;			
before	iteration 1	iteration 2	final
[2] t0 = 2+3	CF: t0 = 5 ;folded from 't0 = 2 + 3'	(t0 gone)	
[2] t1 = t0*4	CP: t1 = 5 * 4 ;t0->5 (CopyProp) DCE: t0 removed (fold note dies)	CF: t1 = 20 ;t0->5; folded from 't1 = 5*4' CP: return 20 ;t1->20 (CopyProp) DCE: t1 removed	(t1 gone) fold Al notes [2] retu

Figure 2. Provenance through the fixed point for $(2+3)*4$: the source line survives every rewrite; transformation notes die twice with DCE'd instructions. Per-instruction provenance is honest but not transitive (§12, item 3).

§7 The Visualization Consumer

The visualizer exists in this paper as *evidence about the architecture*, not as a contribution in the visualization lineage it belongs to — a lineage reaching back to the UW Illustrated Compiler, which displayed its own control and data structures to students in 1988 [28] (§5.5). Its claim is narrow: because explanation is preserved as structured data (§6), building a full-pipeline inspection surface required no new information channels — only renderers.

One CLI mode (`--json`) emits the entire compilation as a single JSON object: tokens with positions, the type-annotated AST, the semantic check log, IR before and after optimization (both as display text and as structured per-instruction records with source line and transformation note), the optimization reports, the final assembly text, and the diagnostics with their recorded traces. The export has a fixed shape: a stage that never ran contributes an explicit empty value, never an absent key, so consumers need no existence logic. A stdlib-only local server shells out to the compiled binary per request; the frontend is framework-free JavaScript rendering one tab per stage (Fig. F6). Stages that did not run say why ("no semantic checks ran — lex/parse stopped first"), which is itself explanation: the pipeline's control-flow decisions are user-visible state.

Two properties matter for the thesis. First, **single source of truth**: the diagnostics tab renders the same struct the CLI renders — same sections, same recorded trace with the same runtime details, same invalid/valid examples —

because both are renderers over one value (D6); divergence between the two surfaces was an actual defect class earlier in this project's history, eliminated by construction rather than by discipline. Second, **provenance becomes navigable**: the IR panels carry per-instruction source-line gutters and optimizer notes from the same fields codegen prints as assembly comments, so "which line did this instruction come from, and what happened to it" is answerable in every representation the compiler has.

The design deliberately avoids claiming more: there is no cross-panel linked highlighting (clicking an assembly line does not select its source token), no time-travel through optimizer iterations, and no user evaluation of the interface — the first two are engineering the exported data already supports (§13), the third is out of scope with the rest of the human questions (§11).

§8 Implementation

8.1 Shape and size

The reference compiler is ~5,943 lines of C++14 (measured at session B; re-measure at submission) with zero external dependencies — the test framework, JSON emission, and visualization server are hand-rolled or stdlib-only, so the artifact builds from a compiler and Make alone. Distribution by subsystem:

subsystem	LOC	subsystem	LOC
diagnostics	1,739	ast (4 visitors)	500
parser	695	main/driver CLI	499
semantic	604	lexer	421
ir	463	codegen	385
optimizer	370	core + toolchain	267

The headline number is the first row: **the diagnostics subsystem is the largest in the compiler — 29.3% of the source** — which is the architecture's priority expressed as a measurement rather than a slogan. (The share has grown across the project: 27% at the initial audit, 28.2% after trace recording, 29.3% after lazy formatting.)

8.2 Mechanism invasiveness (E5)

From the introducing commit's diff, compiler source only: trace recording cost ~200 lines (the recorder, three stage integrations, and surfacing through the renderers), and provenance ~115 lines (IR fields, generator stamping, preservation discipline in two passes, assembly comments, JSON export) — ~320 lines across 17 files, ~5.6% of the then-current source, for both mechanisms. The lazy-formatting repair (D14) later replaced the recorder's internals and touched the 20 scope call sites mechanically.

H1's "localized mechanisms" claim rests on these numbers, with the honest caveat that locality was measured in a codebase designed to receive the mechanisms; §10 discusses what retrofit would change.

8.3 Testing as method

205 tests across nine suites (603 assertions) enforce three distinct kinds of claim. Unit suites pin stage behavior. End-to-end suites refuse to mock: generated assembly is assembled and linked with the real system toolchain, the produced executable is *run*, and its process exit code is asserted — if codegen is wrong, a real binary misbehaves, not an abstraction. Property suites encode the diagnostic-quality invariants of §6 and §9.4, including the trace-accuracy tests whose runtime-fact assertions distinguish recorded from curated traces, and the `--check` silence tests that protect the measurement channel every number in §9 depends on. There is no CI and no coverage measurement — stated as fact; the suite runs on one platform, like everything else here.

8.4 Empirical construction on a legacy target

The target toolchain (GCC 6.3.0, MinGW.org, 32-bit) predates reliable documentation for several of its contracts, so codegen facts were established by probing rather than assumed: the CRT's entry-point requirements (the `_main` symbol and `call __main` sequence) were derived by diffing real `gcc -S` output on probe files after link failures whose messages pointed nowhere near the cause. The same empiricism extended to the measurement harness, where three Windows process-management quirks (a null `ExitCode` without a cached handle; `CreateProcess` rejecting relative forward-slash paths) each produced data that *looked* like compiler behavior until reproduced in isolation — the §9.2 discarded-campaign note is the largest instance of the same lesson. We record these because the paper's numbers pass through this machinery, and a reader repeating the experiments on Windows will meet the same artifacts.

8.5 Availability

The complete source, test suites, corpus generator, measurement and analysis scripts, mutation tooling, raw result CSVs for every campaign (including the discarded and superseded ones), and this paper's working documents are in one public repository, tagged for artifact evaluation at submission. Every number in §9 regenerates from three documented commands.

§9 Evaluation

9.1 Methodology

We evaluate three measurable questions from §3: the invasiveness of the mechanisms (H1/E5), their always-on runtime and memory cost (H3/E1/E2), and the structural-quality properties of the diagnostics they enable (E3/E4). Human benefit is explicitly *not* evaluated (§13 designs that study; §11 explains why the claim is withheld).

Workload. Deterministic, seeded, *valid* programs at 100 / 1,000 / 5,000 / 20,000 lines (many 20–40-statement functions of mixed-depth integer arithmetic). Valid programs are the fair worst case for this architecture: every mechanism pays its cost and no diagnostic ever redeems it.

Measurement path. `--check`: every stage through assembly text, no JSON serialization, no toolchain subprocess, nothing printed on success — the stopwatch sees the compiler and only the compiler. This mode exists because our first campaign timed `--json` and measured the harness instead (§9.2, "a discarded campaign").

Design. Factorial ablation over two compile-time flags (recording, provenance), giving four binaries: FULL, NOTRACE, NOPROV, BASELINE — each mechanism's cost is attributed separately. Ablation is by macro so argument expressions vanish too; a no-op class would understate the cost it exists to measure (§6.3, D13).

Protocol. Per (config, size): 3 warmups, 30 timed runs, 5 peak-working-set runs (sampled live; the counter is unreadable after exit). Binaries built `-O2`. Statistics: median and MAD per cell — wall times on a desktop OS are right-skewed by scheduler and antivirus interference, and medians resist those spikes; overhead is (median_config – median_baseline)/median_baseline with a 95% bootstrap CI (10,000 resamples, percentile method, fixed seed). Alternatives rejected: mean±SD assumes symmetry the data does not have; comparing against production-compiler *speed* would be meaningless at this scale, so all comparisons are self-vs-self.

Environment. Intel Core i7-8665U @ 1.90 GHz, 32 GiB RAM, Windows 11 Pro (build 26200), GCC 6.3.0 (MinGW.org, 32-bit target). Single machine, single toolchain: an external-validity limit stated in §11, not an inconvenience to hide.

9.2 Overhead (E1/E2)

A discarded campaign (methodological note). Our first E1 collection timed `--json` with output piped through the measurement shell. The numbers failed the coherence check we ran before accepting any of them: ablated configurations measured *slower* than the full build,

orderings flipped between sizes, and "baseline" was nine seconds for a 20k-line file through an $O(n)$ pipeline. The harness was ingesting megabytes of serialized pipeline state per run — we had measured PowerShell. The dataset was deleted, the `--check` path built, and the campaign rerun. We report this because measurement pitfalls that are survived silently get repeated by readers.

Naive recorder (eager detail strings) — the result that refuted H3. Median $\pm$ MAD, ms:

lines	baseline	notrace	noprov	full
100	46.9 $\pm$ 2.1	53.4 $\pm$ 2.5	58.3 $\pm$ 3.4	73.8 $\pm$ 4.1
1,000	154.8 $\pm$ 14.9	188.7 $\pm$ 8.8	278.4 $\pm$ 5.2	335.6 $\pm$ 11.4
5,000	570.9 $\pm$ 7.1	824.3 $\pm$ 48.3	1,569.9 $\pm$ 59.0	1,493.6 $\pm$ 53.5
20,000	2,266.0 $\pm$ 133.0	2,237.9 $\pm$ 67.5	5,498.2 $\pm$ 312.7	5,824.2 $\pm$ 221.1

At 20,000 lines — the startup-amortized row — trace recording cost +143% [130, 168] (NOPROV) and the full configuration +157% [144, 204]. The cause is identifiable, not mysterious: the lexer opens two scopes per token, each eagerly building detail strings — order 10^5 – 10^6 heap constructions per clean compile, all discarded. This campaign also *appeared* to show provenance as free (NOTRACE vs BASELINE: -1.2% , CI spanning zero) — a reading the interleaved protocol below corrects; under blocked measurement, whichever config runs nearest the baseline in time inherits flattering drift.

A second protocol iteration. Re-measuring the lazy recorder under the same blocked protocol produced medians that *decrease monotonically in run order* across configs (1890 $\rightarrow$ 1813 $\rightarrow$ 1532 $\rightarrow$ 1512 ms at 20k) — the machine sped up as the campaign progressed, and with mechanism costs now small, that drift dominated the between-config deltas. The final protocol therefore interleaves configs round-robin within each run index, rotating cycle order to cancel position bias. Blocked data is archived, not used.

Lazy recorder (D14), interleaved protocol — the authoritative table. Fig. F7 plots both campaigns' overhead on one shared scale (the flat right panel is the repair); the figure is generated from the raw CSVs by a script that hard-asserts every plotted median against the tables below, so figure and paper cannot silently diverge. Median $\pm$ MAD, ms:

lines	baseline	notrace	noprov	full
100	24.1 $\pm$ 0.9	27.0 $\pm$ 1.9	24.4 $\pm$ 1.0	25.8 $\pm$ 0.8
1,000	78.6 $\pm$ 2.5	89.9 $\pm$ 2.6	79.9 $\pm$ 2.8	90.3 $\pm$ 2.5

5,000	398.4 $\pm$ 12.1	467.2 $\pm$ 11.0	402.7 $\pm$ 12.1	467.8 $\pm$ 10.2
20,000	1,549.5 $\pm$ 76.1	1,801.6 $\pm$ 76.5	1,585.8 $\pm$ 94.7	1,844.5 $\pm$ 94.2

Overhead vs baseline (95% bootstrap CI):

lines	full (total)	trace only (NOPROV)	provenance only (NOTRACE)
100	6.7% [2.2, 11.0]	1.3% [-3.8, 4.0]	11.9% [4.1, 15.5]
1,000	14.9% [10.4, 17.6]	1.6% [-2.1, 4.8]	14.3% [10.2, 17.6]
5,000	17.4% [13.9, 19.9]	1.1% [-2.5, 3.2]	17.2% [14.6, 20.0]
20,000	19.0% [11.8, 28.3]	2.3% [-5.5, 8.3]	16.3% [8.6, 24.5]

Three results, with their cross-campaign caveat stated first: absolute times are not comparable between campaigns (baselines moved with machine state across sessions); every comparison here is *within* one internally-controlled campaign, and the naive-vs-lazy arc compares relative overheads, each measured against its own contemporaneous baseline.

(1) **Lazy recording is statistically free:** 1–2% at every size with every CI spanning zero — down from +143%, a $\sim 60\times$ reduction, achieved with no loss of output (the trace-content tests pin byte-identical details and passed unchanged, 205/205). (2) **Provenance costs a steady 12–17%** — the honest correction of the blocked campaigns' contradictory readings (-1.2% and $+19.9\%$ were both drift). The cost is structural and now grounded, not conjectured: adding span + note grew `sizeof(IRInstruction)` from 116 to 160 bytes (+38%; 20 bytes of span, 24 of MinGW's `std::string`, measured), so every vector element, copy, and pass traversal moves more data, and codegen additionally builds provenance text per instruction. Reduction paths (a compressed span representation; gating comment emission) are future work with a target number attached, not hand-waving: the fields are value-carried by design (D10), and that design's price is measured. (3) At startup-amortized sizes the costs compose additively (20k: 19.0 $\approx$ 2.3 + 16.3; 5k: 17.4 vs 18.3), consistent with independent mechanisms; the 100-line row does *not* compose (6.7 vs 13.2 summed) — at 25 ms total runtime, process startup dominates and the CIs are too wide for attribution, which is why the size ladder exists. The 20k CIs are themselves wide ([11.8, 28.3] for FULL); tightening them means more runs at the largest size, noted as a cheap improvement rather than done, since no conclusion here turns on the difference between 15% and 20%.

Memory (E2). Peak working set is unaffected throughout: $\leq 1.3\%$ delta at every size in every campaign (interleaved, 20k: 85.8 MiB baseline $\rightarrow$ 86.9 MiB full). Open-frame storage and per-instruction fields are memory-negligible; the cost of provenance is time, not space.

9.3 Diagnostic quality (E3/E4)

E3 — structural completeness (component presence, not quality). E3's epistemic role must be stated precisely: it is a structural *inventory*, not a comparison study. That our output contains the sections we designed is near-tautological; the informative content is the contrast *shape* — which explanation components a conventional pipeline emits at all — and the confirmation that no component silently fails to render on any error class. Nine single-error programs through both compilers on identical inputs; mechanical detection rules only (the presence-rubric approach descends from the error-message evaluation instrument of [13], narrowed from judging responses to detecting components). Ours presents location, cause, explanation prose, fix suggestions, trace, and caret on 9/9 (plus invalid/valid examples exactly on the two malformed-expression cases, where the kind defines them); GCC 6.3.0 presents location, cause, and caret on 9/9, and no note, fix-it, example, or trace on any of the nine. Two scope statements keep this honest: the baseline is the 2016 toolchain this compiler targets — no claim is made against current GCC/Clang without running them — and presence is not helpfulness; the enhanced-messages literature is contested on the latter [1, 2], which is precisely why the paper measures the former.

E4 — cascade behavior under single-fault mutation. 100 mutants, each exactly one token-level edit from a valid seed (delete / duplicate / undeclared-substitution at use sites only — declaration sites are excluded because corrupting a declaration creates *k* *genuine* downstream faults and would poison the one-fault ground truth). Error diagnostics per rejected mutant:

compiler	rejected	accepted	distribution
ours	100	0	median 1, mean 1.32, max 5 (73×1, 24×2, 2×3, 1×5)

g++ 6.3	90	10	median 1, mean 1.14, max 2 (77×1, 13×2)
---------	----	----	--

Findings, stated against ourselves first: (1) the shape-specific cascade-freedom invariants do not generalize for free — 3% of mutants still produce ≥ 3 diagnostics, so the tested-invariant approach is necessary but not sufficient, and mutation testing is how the residual frontier gets measured (the discovered shapes are enumerable future fixes); (2) g++ is slightly tighter on this corpus; (3) the accepted sets differ because the *languages* differ (C permits ; ; and { ; this language does not), so the distributions are not over identical mutant sets — an intersection-set statistic is listed as a refinement in §13; (4) all 100 mutants derive from a single 78-line seed, so mutation diversity is bounded by one program's token population — a multi-seed corpus is the other §13 refinement, and no generalization beyond this corpus is claimed. Framing: cascading error reporting is a quantified problem in the recovery literature [6]; our contribution here is the *property-test* treatment, not a recovery algorithm.

E5 — invasiveness. From the introducing commit's diff, compiler source only: trace recording ≈ 200 lines (recorder + three stage integrations + surfacing), provenance ≈ 115 lines (IR fields, generator stamping, two pass-preservation sites, assembly comments, JSON export) — ≈ 320 lines across 17 files, 5.6% of compiler source, supporting H1's "localized mechanisms" claim with a number. The diagnostics subsystem overall was 28.2% of compiler source at the mechanism-introducing commit and 29.3% at the time of writing — §8.1 traces the growth — the architecture's priorities, measured.

9.4 The test suite as evidence

205 tests (603 assertions) function as more than regression cover: the trace-accuracy tests are the *evidence mechanism* for H2 (they assert runtime facts no static chain could contain, and their failure at the curated $\rightarrow$ recorded transition is documented evidence the traces changed nature); the cascade tests pin the quality invariants E4 then stress-tests; the byte-stability of golden IR tests across the provenance change demonstrates the compatibility cost D10 accepted; and the --check silence tests protect the measurement channel every number in §9.2 depends on.

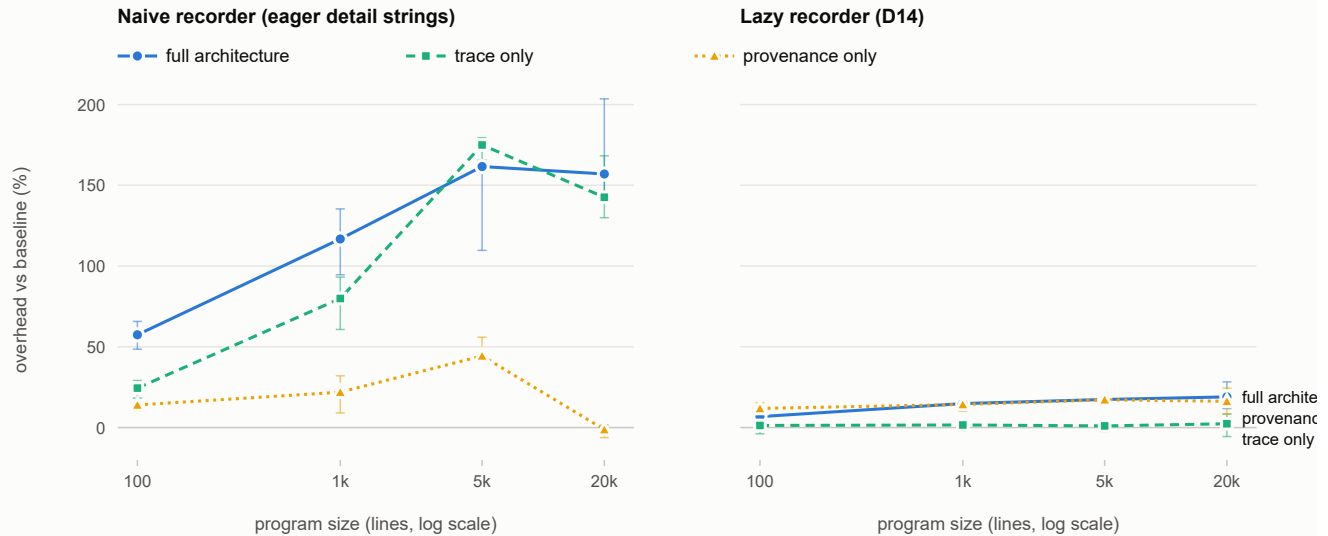


Figure 3. Always-on overhead vs the ablated baseline, naive (left) and lazy (right) recorders, shared scale; whiskers are 95% bootstrap CIs. Generated by a script that hard-asserts every plotted median against the tables in §9.

§10 Discussion

10.1 Where the knowledge base stops scaling

The switch-indexed knowledge base (D4) is honest about its ceiling, and we hit the first warning shot ourselves: the `detail/detail2` parameterization ran out of arity the first time a diagnostic needed three runtime values, and the factory works around it by re-deriving one field (§6.2). At seventeen kinds the workaround is a documented oddity; at hundreds it would be a code smell; at Clang's or Rust's scale it is exactly why those projects hold messages in TableGen and Fluent respectively [10, 27]. The pattern's load-bearing property — one place where all prose lives, indexed by root cause — survives that migration; what changes is the storage (data files instead of a switch) and the parameterization (named placeholders instead of positional details). We read our capacity limit as evidence *for* the industrial designs, reached from the opposite direction.

10.2 Trace depth in bigger languages

Our recorded traces are short because the language is flat: no recursion in the grammar beyond expressions, no template instantiation, no macro expansion. In a production front end the active-stack snapshot could be hundreds of frames deep (consider a C++ template error's instantiation stack — which production compilers already summarize, badly or well, because they face the same problem). The scope-placement rule (D9) generalizes — scopes where diagnostics can fire, plus structural landmarks — but a production adoption would need a *budget policy* the rule does not yet have: depth windows, frame sampling, or collapse-repeated-frames

summarization. We flag this as the first question a scaling attempt must answer, not as a solved problem.

10.3 Provenance's price in context

The 12–17% always-on cost (§9.2) looks alarming until placed against practice: production compilers already carry per-instruction source locations when building with debug info, and LLVM's remark infrastructure [5] attaches transformation records opt-in. Our number is what *always-on, value-carried* provenance costs in a compiler whose instructions were previously 116 bytes of pure computation — a worst-case framing, since any compiler with existing debug-location machinery has already paid most of the memory-layout bill. The measured reduction paths are concrete: packing the span (20 bytes of four ints could be an interned 4-byte index), interning note strings, or gating note construction the way recording gates detail construction (D14 — the same fix, one field over). We did not apply them because 12–17% at teaching scale does not obstruct the thesis, and an optimization pass without a driving requirement is how measurement artifacts get built.

10.4 Two observability modes, one architecture

The implementation carries both a *narrating* channel (the semantic check log: one line per check performed, always on, educational) and an *evidentiary* channel (recorded traces: silent until a diagnostic fires). They answer different questions — "what is the compiler doing?" versus "why did THIS happen?" — and their coexistence is not redundancy but a division the architecture makes cheap: both are consumers of the same stage-owned state. The duality suggests the general shape of compiler observability:

continuous narration for learning contexts, on-demand evidence for diagnosis, one contract underneath.

10.5 Retrofit versus designed-in

GHC's and Rust's structured-diagnostics migrations [4, 9, 3] are the strongest external evidence this architecture addresses a real need — and the strongest evidence about cost asymmetry. Their retrofits are multi-year, multi-release migrations across entrenched string-based call sites [4, 9]; our designed-in version cost ≈ 320 lines and 5.6% of a (small) codebase (§8.2). The honest generalization is not "designing in is cheap" — our codebase was shaped to receive it — but narrower: the *marginal* cost of the explanation contract is small when the failure-handling architecture is decided early, and grows with every string-typed diagnostic call site added before the decision.

10.6 Evidence for the LLM era

Generated error explanations are being deployed now, with mixed results [25, 26]. Whatever position one takes on wording generation, an LLM explainer needs *grounding* — the actual construct, the actual path, the actual transformation history — precisely the evidence conventional pipelines discard and this architecture preserves as structured values. We speculate, and mark it as speculation, that recorded traces and provenance form a natural retrieval substrate for grounded generation; nothing in this paper tests that.

§11 Threats to Validity

Construct validity. The central operationalization is stated in bold because it bounds every quality claim: **component presence is not helpfulness.** E3 detects whether explanation components exist, by author-defined mechanical rules; whether they help anyone is a human question this paper does not answer, and the literature warns loudly against assuming it [1, 2]. Likewise "explanation" is operationalized as preserved structure (spans, traces, provenance, root causes) — a reader who means something richer by the word should read our claims as being about the structure. The E4 cascade metric assumes one token edit equals one fault; the mutation operators were restricted (declaration sites excluded) specifically to protect that assumption, but the assumption itself is a modeling choice.

Internal validity. The ablation compares four *binaries*, so config deltas fold in code-layout differences, not only mechanism work; two observations bound the concern — the NOPROV-vs-BASELINE deltas have CIs spanning zero at every size (a layout effect would have to be conveniently near-null four times), and the provenance delta has an

independent structural witness (the +38% instruction size, §9.2). Temporal drift dominated one whole campaign before the interleaved protocol; interleaving spreads residual drift across configs but cannot remove it. The bootstrap uses a fixed seed (reproducibility over seed-sensitivity; re-running with other seeds is one flag). The measurement harness itself produced a family of platform artifacts before yielding trustworthy data — the process-management quirks of §8.4 and the measurement-channel contamination of the discarded campaign (§9.2) — each found by coherence checking; we cannot rule out another we did not trigger.

External validity. One tiny language, one implementation, one machine, one OS, one 2016 toolchain. The corpus is author-generated, arithmetic-heavy, and error-free by design (it measures always-on cost, not diagnostic-path cost); the mutant population derives from a single seed program; the E3 baseline is the GCC the compiler targets, not current GCC or Clang. Every generalization beyond this artifact is either explicitly argued in §10 or not made.

Conclusion validity. 30 runs per timing cell; medians with MAD; bootstrap CIs reported wherever a comparison is made. The widest intervals (20k, FULL: [11.8, 28.3]) are printed next to the claims they qualify. Where a result contradicted physical sense — orderings flipping, ablated configs "slower" — we discarded the data and said so, rather than narrating it.

§12 Limitations

Stated as facts about the artifact, each with its consequence.

1. **The curated fallback exists.** Diagnostics constructed through an engine with no recorder attached carry hand-written reference chains, not recorded ones. In the compiled pipeline this path is unreachable (verified by exhaustive search: three engines, three constructor-attached recorders, no other instantiation), but the dual source of truth remains a maintenance hazard bounded only by tests.
2. **Scope placement is selective.** Recorded traces are coarse to the granularity of the placement rule (D9); omission cannot fabricate a path, but a frame the rule skips is a frame the user never sees.
3. **Per-instruction provenance is not transitive** under dead-code elimination (§6.4): a folded instruction's history dies with it once propagation makes it dead. Locked in by tests as documented semantics; a dataflow-chained design is future work.
4. **Five diagnostic kinds are reserved-unreachable** — defined, fully worded, never emitted, because the

language cannot yet produce them. Tests are forbidden from expecting them; counting them as supported would inflate the taxonomy by 29%.

5. **The cascade invariants are shape-specific.** Mutation testing measured the residual: 3% of single-fault mutants still produce three or more diagnostics (§9.3).

6. **Assembly provenance is comments, not debug info.** Readable by humans, invisible to debuggers; no DWARF/.loc emission.

7. **The knowledge base centralizes prose but implements no internationalization** — centralization makes it possible (one place to translate), and we did none of it.

8. **Single translation unit, no calls, no control flow.** The cascade and trace-depth results inherit the language's flatness; §10.2 discusses what deeper languages would demand.

9. **The recorder is single-threaded by construction** — one recorder per stage, no synchronization; a parallel front end would need per-thread recorders and a merge discipline.

10. **Windows-locked build and measurement.** The Makefile's shell commands, the harness, and all numbers are from one platform (§8.4, §11).

§13 Future Work

13.1 The user study (designed, not executed)

Per the claim discipline of §§3, 9, and 11, no human-benefit claim appears in this paper. The study that could license one is designed here so that running it is an execution task, not a research-design task.

Research question. Does the full explanation format reduce time-to-correct-fix and improve cause-identification accuracy for novice programmers, relative to conventional single-line diagnostics — and which components (trace, examples, fixes) do participants actually consult?

Built-in manipulation. The compiler ships both renderers over the same diagnostic values (§6.2, D6): the full sectioned format and the compact one-liner. The experimental conditions therefore differ *only* in presentation — same compiler, same detection, same spans — which is precisely the isolation the contested prior studies struggled to guarantee [1, 2].

Design. Between-subjects (a within-subjects design would leak explanation content across conditions), participants from CS1/CS2 populations, block-randomized by course section. Task corpus: the nine E3 error programs plus a curated sample of E4 mutants, ordered by a fixed schedule.

Measures: (i) time from first diagnostic display to first correct fix (compile-verified); (ii) cause-identification accuracy on a short structured response, scored by the rubric tradition of [13]; (iii) component-consultation behavior via screen recording (which sections are visibly read); (iv) optional self-report workload. Pilot with 10–15 participants to estimate variance; power the main study from the pilot (the literature's inconclusive results [2] make an a-priori effect-size assumption indefensible — that is the lesson of the enhanced-messages saga). Pre-register hypotheses and analysis (mixed-effects models; participant and task as random effects). Threats to plan for: novelty effects favoring the richer format, instructor enthusiasm bias, and the generalization limits of a kernel language — all to be reported, not managed away.

13.2 Architecture extensions

- **Transitive provenance.** Chain notes through value flow so a surviving instruction inherits the history of the dead instructions that produced its operands — erasing limitation 3 at the cost of note growth that must itself be measured (the D14 lesson: gate construction, measure before shipping).
- **Trace budget policies** for deep languages (§10.2): depth windows, repeated-frame collapse, sampling — each a measurable tradeoff between trace fidelity and the overhead E1 bounds.
- **Provenance cost reduction** with a target: pack the span, intern notes, gate note construction; the 12–17% figure (§9.2) is the number to beat, the +38% instruction growth the lever.
- **Knowledge-base externalization experiment:** migrate the switch to data files at the point positional details break again (§10.1), measuring what the industrial DSL layer actually buys.

13.3 Evaluation extensions

- **Multi-seed mutation corpus** and the **intersection-set statistic** (limitations of E4, §9.3); plus mutation operators derived from real student error distributions [14] rather than token edits alone.
- **Tighter 20k confidence intervals** (more runs; cheap).
- **A modern-baseline E3** against current GCC/Clang — the honest comparison this paper deliberately did not claim (§9.3).

13.4 Consumers

- **LSP adapter** over the JSON export — structured diagnostics, fixes, and traces are already in machine shape (§7); the adapter is engineering, and would test

the export contract against a consumer we did not design.

- **Linked highlighting** in the visualizer (assembly line → IR instruction → source token), for which the provenance fields already carry the joins.
- **Grounded generation**: the speculative §10.6 pairing of recorded evidence with LLM explainers [25, 26], which would need its own evaluation under exactly the discipline of §13.1.

§14 Conclusion

This work began with a compiler that detected a defect and told no one — its analyzer logged the finding, its user got a green build and a garbage exit code. The paper's answer is architectural: treat explanation as an obligation of every stage, and make discarding evidence a type error rather than a habit. In the resulting design, every diagnostic is a structured value whose trace was recorded from the compiler's actual execution (C2), assembled by a five-role core whose taxonomy doubles as its explanation index (C1), over a pipeline that carries source provenance all the way into emitted assembly (C3), with the properties that make diagnostics trustworthy enforced as executable tests and stress-measured by mutation (C4).

The evidence is a complete, zero-dependency reference compiler and a measurement campaign that pushed back hard enough to be believed: one dataset discarded for measuring the harness, one hypothesis refuted — naive recording cost $2.4\times$ — and repaired by a lazy design to statistical insignificance, and one comfortable assumption corrected, since provenance is not free but a steady, mechanically explained 12–17%. The negative results — non-transitive provenance under dead-code elimination, a 3% residual cascade the invariant tests do not cover — are reported as findings because they delimit what this architecture can promise.

What we hope a reader takes away is smaller and sharper than "compilers should explain themselves": that the evidence for explanation is present in every compiler at the moment of failure; that retaining it is an architectural decision with a measurable, modest price when made early; and that the difference between a compiler that knew and a compiler that says is, concretely, about three hundred lines of discipline.

References

1. P. Denny, A. Luxton-Reilly, D. Carpenter. "Enhancing Syntax Error Messages Appears Ineffective." ITiCSE 2014, pp. 273-278. DOI 10.1145/2591708.2591748.
2. R. Pettit, J. Homer, R. Gee. "Do Enhanced Compiler Error Messages Help Students? Results Inconclusive." SIGCSE 2017. DOI 10.1145/3017680.3017768.
3. Rust RFC 1644, "Default and expanded rustc errors." <https://rust-lang.github.io/rfcs/1644-default-and-expanded-rustc-errors.html>
4. GHC development documentation: "Errors as (structured) values" (GHC wiki) and issue #18516 "Move to a more structured error representation." gitlab.haskell.org/ghc/ghc.
5. LLVM Project. "Remarks" (llvm.org/docs/Remarks.html) and Clang User's Manual, `-fsave-optimization-record`; `llvm-opt-report`.
6. L. Diekmann, L. Tratt. "Don't Panic! Better, Fewer, Syntax Errors for LR Parsers." ECOOP 2020, LIPIcs. DOI 10.4230/LIPIcs.ECOOP.2020.6 (also arXiv:1804.07133).
7. LLVM Project. "LLVM Programmer's Manual" — error handling: recoverable errors, `Expected<T>/Error`. <https://llvm.org/docs/ProgrammersManual.html>
8. The Haskell Error Index. <https://errors.haskell.org> (error codes [GHC-12345] since GHC 9.6.1).
9. Well-Typed. "The new GHC diagnostic infrastructure" (2021). <https://well-typed.com/blog/2021/08/the-new-ghc-diagnostic-infrastructure/>
10. Rust Compiler Development Guide, diagnostics chapter (structured suggestions API; Fluent-based translation) and UI-tests chapter (`stderr` golden snapshots). rustc-dev-guide.rust-lang.org/diagnostics.html, [/tests/ui.html](https://rustc-dev-guide.rust-lang.org/ui-tests.html)
11. A. J. Ko, B. A. Myers. "Debugging Reinvented: Asking and Answering Why and Why Not Questions about Program Behavior." ICSE 2008. DOI 10.1145/1368088.1368130. (ICSE 2018 Most Influential Paper.)
12. The rustc book, "JSON Output." <https://doc.rust-lang.org/rustc/json.html>
13. G. Marceau, K. Fisler, S. Krishnamurthi. "Measuring the Effectiveness of Error Messages Designed for Novice Programmers." SIGCSE 2011, pp. 499-504. DOI 10.1145/1953163.1953308. (Best Paper.)
14. B. A. Becker, P. Denny, R. Pettit, D. Bouchard, D. J. Bouvier, B. Harrington, A. Kamil, A. Karkare, C. McDonald, P.-M. Osera, J. L. Pearce, J. Prather. "Compiler Error Messages Considered Unhelpful: The Landscape of Text-Based Programming Error Message Research." ITiCSE-WGR 2019. DOI 10.1145/3344429.3372508.
15. T. Barik, J. Smith, K. Lubick, E. Holmes, J. Feng, E. Murphy-Hill, C. Parnin. "Do Developers Read Compiler Error Messages?" ICSE 2017. DOI 10.1109/ICSE.2017.59.
16. P. Denny, J. Prather, B. A. Becker, C. Mooney, J. Homer, Z. C. Albrecht, G. B. Powell. "On Designing Programming Error Messages for Novices: Readability and its Constituent Factors." CHI 2021. DOI 10.1145/3411764.3445696.
17. D. Zhang, A. C. Myers. "Toward General Diagnosis of Static Errors." POPL 2014, pp. 569-582. DOI 10.1145/2535838.2535870.
18. D. Zhang, A. C. Myers, D. Vytiniotis, S. Peyton-Jones. "SHerrLoc: A Static Holistic Error Locator." ACM TOPLAS 39(4), Article 18, Aug 2017. DOI 10.1145/3121137.

19. D. Zhang, A. C. Myers, D. Vytiniotis, S. Peyton Jones. "Diagnosing Type Errors with Class." PLDI 2015. DOI 10.1145/2737924.2738009.
20. Z. Pavlinovic, T. King, T. Wies. "Practical SMT-Based Type Error Localization." ICFP 2015. DOI 10.1145/2784731.2784765.
21. A. J. Ko, B. A. Myers. "Extracting and Answering Why and Why Not Questions about Java Program Output." ACM TOSEM 20(2), Article 4, Aug 2010. DOI 10.1145/1824760.1824761.
22. F. J. Almeida-Martínez, J. Á. Velázquez-Iturbide (+ possibly J. Urquiza-Fuentes). "VAST: Visualization of Abstract Syntax Trees within Language Processors Courses." ACM SoftVis 2008. DOI 10.1145/1409720.1409759.
23. D. Rodríguez-Cerezo, M. Gómez-Albarrán, J.-L. Sierra-Rodríguez. "Interactive educational simulations for promoting the comprehension of basic compiler construction concepts." ITiCSE 2013. DOI 10.1145/2462476.2462498.
24. K. Dalbo, Y. Ahmed, H. Lim. "JITScope: Interactive Visualization of JIT Compiler IR Transformations." arXiv:2505.21599 (2025). Preprint — cite as such.
25. A. Taylor, A. Vassar, J. Renzella, H. Pearce. "dcc --help: Transforming the Role of the Compiler by Generating Context-Aware Error Explanations with Large Language Models." SIGCSE 2024. DOI 10.1145/3626252.3630822.
26. E. A. Santos, B. A. Becker. "Not the Silver Bullet: LLM-enhanced Programming Error Messages are Ineffective in Practice." UKICER 2024 (Best Paper). DOI 10.1145/3689535.3689554 (also arXiv:2409.18661).
27. Clang project. "'Clang' CFE Internals Manual" — the Diagnostics subsystem: kinds defined in clang/Basic/Diagnostic*Kinds.td, TableGen-generated IDs/severities/format strings. <https://clang.llvm.org/docs/InternalsManual.html>
28. K. Andrews, R. R. Henry, W. K. Yamamoto. "Design and Implementation of the UW Illustrated Compiler." PLDI 1988 (SIGPLAN Notices 23), pp. 105-114. DOI 10.1145/960116.54001.
29. A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman. "Compilers: Principles, Techniques, and Tools," 2nd ed., Addison-Wesley, 2006. ISBN 978-0321486813.
30. "TraceDiff: Debugging Unexpected Code Behavior Using Trace Divergences." VL/HCC 2017.